

Laboratorio 3 – Algoritmos de Enrutamiento

Descripción de la práctica

El Laboratorio 3 tiene como objetivo implementar una red simulada utilizando un protocolo de enrutamiento **Link State**. Cada router conoce inicialmente únicamente a sus vecinos directos y, mediante el intercambio de mensajes de control, obtiene información suficiente para construir una representación de la topología completa. Los protocolos de estado de enlace se basan precisamente en la distribución de información sobre las conexiones de cada router, permitiendo que los nodos mantengan una base de datos de estado de enlace y calculen rutas de menor costo de manera independiente (Moy, 1998).

La implementación se divide en un **plano de control** y un **plano de datos**. El plano de control utiliza mensajes HELLO y HELLO_ACK para verificar la disponibilidad de los vecinos, mientras que los mensajes LSA (*Link State Advertisement*) distribuyen las conexiones y costos conocidos mediante un algoritmo de *flooding*. Con la información recibida se construye una *Link-State Database* (LSDB) y posteriormente se ejecuta el algoritmo de Dijkstra para generar la tabla nodo_tabla_enrutamiento.csv. Esta arquitectura corresponde a los requerimientos establecidos para el laboratorio.

Una vez estabilizadas las rutas, el plano de datos permite enviar mensajes entre un cliente y un servidor a través de los routers. Los paquetes son serializados, codificados utilizando **Hamming (7,4)** y enviados mediante sockets TCP. Cada router intermedio decodifica el mensaje, corrige un posible error individual, determina el siguiente salto mediante su tabla de enrutamiento y vuelve a codificarlo antes de transmitirlo. Hamming (7,4) utiliza siete bits para representar cuatro bits de información y permite corregir un error de un bit por bloque (MIT OpenCourseWare, 2013). El flujo solicitado de decodificación consulta de la tabla y recodificación en cada salto está establecido explícitamente en el enunciado.

La comunicación entre procesos se realiza mediante **TCP**, protocolo que proporciona un flujo confiable y ordenado de bytes (Eddy, 2022). Para las pruebas locales pueden utilizarse diferentes puertos sobre localhost; posteriormente, la integración entre las tres parejas se realizará utilizando las direcciones privadas proporcionadas por **Tailscale**, tal como establece el laboratorio. Tailscale funciona únicamente como medio de conectividad entre los equipos, mientras que la aplicación continúa comunicándose mediante sockets TCP.

Descripción de los algoritmos utilizados

El primer mecanismo implementado corresponde al **descubrimiento y monitoreo de vecinos**. Cada router envía periódicamente mensajes HELLO hacia los vecinos configurados y espera mensajes HELLO_ACK como confirmación. Si un vecino deja de responder durante el tiempo establecido, el enlace se considera inactivo y el router genera una nueva versión de su LSA. Los protocolos Link State reales, como OSPF, también utilizan mensajes Hello para descubrir y mantener relaciones entre routers vecinos (Moy, 1998).

Para distribuir la información topológica se utiliza el algoritmo de **flooding**. Cada LSA contiene el nodo que lo originó (origin), un número de secuencia (seq) y los enlaces activos con sus respectivos costos. Cuando un router recibe un LSA nuevo, lo almacena en su LSDB y lo reenvía hacia todos sus vecinos activos excepto aquel desde el cual fue recibido. Si el número de secuencia es igual o menor al almacenado, el paquete se descarta. Esta estrategia evita procesar repetidamente información antigua y permite propagar los cambios de topología por toda la red. El formato y comportamiento de estos mensajes se encuentran establecidos en el protocolo definido previamente por el grupo.

Una vez actualizada la LSDB se ejecuta el **algoritmo de Dijkstra**, encargado de encontrar los caminos de menor costo desde el router actual hacia los demás nodos. Dijkstra permite obtener caminos mínimos en grafos con pesos no negativos, condición que se cumple debido a que los costos de los enlaces del protocolo son positivos (MIT OpenCourseWare, 2011). El resultado se transforma en una tabla que almacena el destino, siguiente salto, dirección IP, puerto y costo total. Es importante que la IP y el puerto pertenezcan al **siguiente salto**, ya que cada router únicamente necesita conocer el vecino al cual deberá reenviar el paquete.

Finalmente, para el plano de datos se utiliza **Hamming (7,4)**. Los datos se dividen en grupos de cuatro bits y se agregan tres bits de paridad, generando bloques de siete bits. En cada router se calcula el síndrome del bloque recibido para determinar si existe un error individual y, de ser necesario, corregirlo antes de recuperar los cuatro bits originales. Después de reconstruir el mensaje JSON, el router consulta el destino en su tabla, incrementa el contador de saltos y vuelve a realizar la codificación antes de enviarlo al siguiente nodo. De esta forma, el enrutamiento y la corrección de errores se realizan independientemente en cada salto.

Implementación de los algoritmos

La implementación se basa en un protocolo educativo de enrutamiento Link State ejecutado sobre conexiones TCP. Cada router inicia con una lista de vecinos configurados, incluyendo su dirección IP, puerto y costo del enlace. Periódicamente envía mensajes HELLO y espera recibir HELLO o HELLO_ACK para confirmar que el vecino está disponible. Los vecinos pueden encontrarse en los estados UNKNOWN, UP o DOWN. Si un vecino deja de responder durante el intervalo configurado, se marca como DOWN; cuando vuelve a responder, se marca nuevamente como UP. Cada cambio en el estado de un enlace provoca la generación de un nuevo anuncio de estado de enlace.

Los anuncios LSA contienen el router originador, un número de secuencia y la lista de vecinos activos junto con sus costos. Estos anuncios se distribuyen mediante flooding: cuando un router recibe un LSA nuevo, lo valida, lo almacena en su LSDB y lo reenvía a sus demás vecinos, excepto al vecino que se lo envió. El número de secuencia permite identificar versiones nuevas y evitar procesar o reenviar anuncios repetidos o antiguos. Además, el campo from se actualiza en cada salto para identificar el vecino inmediato que transmitió la copia. De esta forma, cada router construye progresivamente una visión común de la topología de la red.

Para calcular las rutas se utiliza Dijkstra sobre el grafo formado con la información almacenada en la LSDB. Cada router representa un nodo y cada enlace contiene un costo positivo configurable. El algoritmo mantiene una distancia mínima conocida para cada nodo y utiliza una cola de prioridad para seleccionar continuamente el nodo con menor costo acumulado. También se almacena el primer router visitado desde el nodo local, que corresponde al next hop utilizado para reenviar los paquetes. Si existen varias rutas con el mismo costo, se elige de manera determinista el siguiente salto cuyo node_id sea lexicográficamente menor. El resultado se mantiene en memoria y se persiste en un archivo CSV con el destino, siguiente salto, IP, puerto y costo total.

El plano de datos utiliza mensajes JSON con los campos from, to, hops y payload. Antes de enviarse entre routers, el JSON se convierte a bytes UTF-8, luego a bits y finalmente se codifica con Hamming (7,4). Cada bloque de cuatro bits genera una palabra de siete bits formada por cuatro bits de datos y tres bits de paridad par. Al recibir una trama, el router calcula el síndrome de Hamming para detectar la posición de un posible error individual, corrige el bit afectado y extrae nuevamente los datos originales. Después incrementa el contador de saltos, determina si el paquete debe entregarse localmente o reenviarse y vuelve a codificar el mensaje antes de enviarlo al siguiente router. Esto garantiza que Hamming se aplique independientemente en cada salto.

El sistema utiliza framing explícito mediante el carácter de nueva línea (LF), ya que TCP funciona como un flujo continuo de bytes y no conserva los límites de los mensajes. Cada conexión mantiene un buffer que permite reconstruir mensajes divididos entre varias operaciones recv() y procesar varios mensajes recibidos en una sola lectura. La concurrencia se organiza mediante threads, eventos de apagado y locks para proteger la LSDB, los vecinos activos y la tabla de rutas. Los routers, clientes y servidores se ejecutan como procesos independientes y se configuran

mediante archivos JSON, por lo que la misma implementación puede utilizarse tanto con 127.0.0.1 como con direcciones privadas de Tailscale.

Aplicación ATM

Se implementó un ATM y un servidor bancario. El cliente ATM envía una solicitud de autenticación, consulta de saldo o retiro al router gateway. El banco procesa la solicitud y construye una respuesta cuyo destino es el host atm1. El router destino entrega la respuesta al listener TCP del ATM.

Las credenciales incluidas para la prueba son:

Cuenta 1:

- Tarjeta: 123456789
- PIN: 1234

Cuenta 2:

- Tarjeta: 987654321
- PIN: 4321

El protocolo ATM distingue solicitudes y respuestas mediante los tipos ATM_REQUEST y ATM_RESPONSE. El servidor valida la estructura de la solicitud antes de acceder a la cuenta bancaria.

Resultados

Router A

Imagen 1. Conexión del router A con el router B

```
2026-08-18 22:46:39 [DEBUG] [A] HELLO -> B
2026-08-18 22:46:39 [INFO] [A] [LSA] accepted origin=B seq=2
2026-08-18 22:46:39 [INFO] [A] [DIJKSTRA] table updated entries=0
2026-08-18 22:46:39 [INFO] [A] [NEIGHBOR] B -> UP
2026-08-18 22:46:39 [INFO] [A] [LSA] generated seq=2
2026-08-18 22:46:39 [INFO] [A] [DIJKSTRA] table updated entries=1
2026-08-18 22:46:42 [DEBUG] [A] HELLO -> B
2026-08-18 22:46:45 [DEBUG] [A] HELLO -> B
2026-08-18 22:46:48 [DEBUG] [A] HELLO -> B
2026-08-18 22:46:51 [DEBUG] [A] HELLO -> B
2026-08-18 22:46:54 [DEBUG] [A] HELLO -> B
2026-08-18 22:46:57 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:00 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:03 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:06 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:09 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:12 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:14 [INFO] [A] [DATA] bank1 via B
2026-08-18 22:47:15 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:18 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:21 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:24 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:27 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:30 [DEBUG] [A] HELLO -> B
2026-08-18 22:47:33 [DEBUG] [A] HELLO -> B
```

Tabla de enrutamiento A

Destino	Siguiente_salto	IP	Puerto	Costo
B	B	100.92.241.72	5000	1

En la tabla A se observó una ruta hacia B con costo 1 y puerto 5000 para el siguiente salto. Esto representa el puerto de escucha de B en la configuración de la topología correspondiente al archivo generado.

Router B

Imagen 2. Conexión del router B con el router A, proceso de transacción realizada

```

2026-08-18 22:46:39 [INFO] [B] [NEIGHBOR] A -> UP
2026-08-18 22:46:39 [INFO] [B] [LSA] generated seq=2
2026-08-18 22:46:39 [INFO] [B] [DIJKSTRA] table updated entries=1
2026-08-18 22:46:39 [INFO] [B] [LSA] accepted origin=A seq=2
2026-08-18 22:46:39 [INFO] [B] [DIJKSTRA] table updated entries=1
2026-08-18 22:46:40 [DEBUG] [B] HELLO -> A
2026-08-18 22:46:43 [DEBUG] [B] HELLO -> A
2026-08-18 22:46:46 [DEBUG] [B] HELLO -> A
2026-08-18 22:46:49 [DEBUG] [B] HELLO -> A
2026-08-18 22:46:52 [DEBUG] [B] HELLO -> A
2026-08-18 22:46:55 [DEBUG] [B] HELLO -> A
2026-08-18 22:46:58 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:01 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:04 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:07 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:10 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:13 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:14 [INFO] [B] [DATA] atm1 via A
2026-08-18 22:47:16 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:19 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:22 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:25 [DEBUG] [B] HELLO -> A
2026-08-18 22:47:28 [DEBUG] [B] HELLO -> A

```

Servidor

Imagen 3. Vista de procesos en el servidor

```

=====
INICIANDO SERVIDOR ATM
=====

2026-08-18 22:46:49 [INFO] [BANK] Banco escuchando en 100.92.241.72:6001
2026-08-18 22:47:14 [INFO] [BANK] conexión recibida desde ('100.92.241.72', 55118)
2026-08-18 22:47:14 [INFO] [BANK] Solicitud AUTH recibida desde atm1
2026-08-18 22:47:14 [INFO] [BANK] Respuesta enviada hacia atm1
2026-08-18 22:47:39 [INFO] [BANK] conexión recibida desde ('100.92.241.72', 58721)
2026-08-18 22:47:39 [INFO] [BANK] Solicitud BALANCE recibida desde atm1
2026-08-18 22:47:39 [INFO] [BANK] Respuesta enviada hacia atm1
2026-08-18 22:47:46 [INFO] [BANK] conexión recibida desde ('100.92.241.72', 58737)
2026-08-18 22:47:46 [INFO] [BANK] Solicitud WITHDRAW recibida desde atm1
2026-08-18 22:47:46 [INFO] [BANK] Respuesta enviada hacia atm1

```

Cliente

Imagen 4. Menú del ATM en consola, saldo disponible y retiro de dinero en cuenta existente

```
Ingrese número de tarjeta: 2026-08-18 22:47:06 [INFO] [ATM] escuchando respuestas en 100.92.241.72:6000
123456789
Ingrese PIN: 1234

Autenticación exitosa.

1. Consultar saldo
2. Retirar dinero
3. Salir

Seleccione una opción: 1

Saldo disponible: Q5000.00

1. Consultar saldo
2. Retirar dinero
3. Salir

Seleccione una opción: 2
Ingrese monto a retirar: Q2500

Retiro realizado correctamente.
Saldo restante: Q2500.00

1. Consultar saldo
2. Retirar dinero
3. Salir
```

Se puede observar que ambos procesos tanto de consultar saldo disponible como retirar una cantidad de dinero, se realizaron correctamente.

Discusión

Al inicio, realizar las pruebas mostró algunas dificultades debido a las configuraciones iniciales requeridas. Por un lado, se tuvo que realizar la configuración de la red de Tailscale, identificando correctamente cada máquina por su IP. Luego de esto se hicieron pruebas de conexión a las computadoras utilizando TestNet en powershell para ver que la IP y el puerto estuvieran correctas. A partir de esto se realizó la configuración de los archivos correspondientes para poder importar las variables de entorno a los .json

Después de realizar las configuraciones correspondientes, se levantaron los servicios necesarios, comenzando por el Router A, luego el Router B. Luego se levantaron los servicios del cliente del ATM y el Servidor. Al ver que todo estuviera conectado, se realizaron las operaciones correspondientes, ingresando el número de tarjeta, el pin, retiro del dinero y consulta de saldo.

Al realizar una acción, era posible visualizarla en lo que era el servidor, lo que validaba que se estuvieran realizando las operaciones deseadas.

El diseño por capas facilita el diagnóstico. El framing y Hamming pueden probarse sin una red física, mientras que la resolución de hosts y el forwarding necesitan routers activos. La separación

entre control, forwarding y entrega de hosts también permite que el router mantenga la convergencia de rutas mientras procesa datos.

Conclusiones

- Se implementó un protocolo Link State con descubrimiento de vecinos, flooding de LSAs, LSDB y cálculo de rutas mediante Dijkstra.
- El plano de datos resuelve hosts a gateways y reenvía mensajes usando el siguiente salto
- El diagnóstico del ATM mostró que una discrepancia de un solo puerto puede impedir todo el flujo de aplicación, aunque el código de routing esté funcionando.

Comentarios

Para repetir la prueba se deben iniciar primero los routers, esperar la convergencia de vecinos y rutas, iniciar el servidor ATM y finalmente iniciar el cliente ATM. Para realizar las pruebas se deben utilizar los datos de las cuentas definidas en la sección “Aplicación ATM”.

Referencias

Bray, T. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259)*. Internet Engineering Task Force. RFC Editor.

Eddy, W. (Ed.). (2022). *Transmission Control Protocol (TCP) (RFC 9293)*. Internet Engineering Task Force. RFC Editor.

MIT OpenCourseWare. (2011). *Lecture 16: Dijkstra*. Massachusetts Institute of Technology.

MIT OpenCourseWare. (2013). *Principles of discrete applied mathematics: Hamming codes*. Massachusetts Institute of Technology.

Moy, J. (1998). *OSPF Version 2 (RFC 2328)*. Internet Engineering Task Force. RFC Editor.